

ENGINEERING STRATEGY · VERSION 2.0

MetaRDU Desktop Upgrade Plan

A comprehensive 9-month roadmap to transform MetaRDU Desktop from a surveyor's office tool into a production-grade, Tauri-based drone survey workstation — with flight planning, live drone link, in-app photogrammetry, real raster I/O, and ML feature extraction.

9

MONTHS

3

PHASES

5

DRONE CAPABILITIES

6

MATH STANDARDS

Prepared for **error302**

Repositories **metardu-desktop** · **metardu** · **agency-agents**

Date **July 2026**

PRODUCTION-READINESS PLAN

CONFIDENTIAL · INTERNAL STRATEGY

Table of Contents

Chapter 1: Executive Summary	4
Chapter 2: Current State Assessment	5
2.1 Tech Stack	5
2.2 Three Surveying Verticals	6
2.3 Drone Capabilities: Present vs Absent	6
2.4 Critical Gaps Summary	7
Chapter 3: Vision, Goals, and Success Metrics	8
3.1 Five Strategic Goals	8
3.2 Success Metrics	8
Chapter 4: Architecture Decisions (ADRs 006-012)	10
4.1 ADR-006: Rust Sidecar Strategy (Phase 1-2)	10
4.2 ADR-007: Tauri 2.x Migration (Phase 3)	10
4.3 ADR-008: Drone Protocol Stack	10
4.4 ADR-009: Photogrammetry Sidecar (OpenDroneMap)	11
4.5 ADR-010: GDAL Integration	11
4.6 ADR-011: ML Pipeline (ONNX Runtime)	11
4.7 ADR-012: IPC Validation (Zod Schemas)	12
Chapter 5: Phased Roadmap (3 Phases × 3 Months)	12
5.1 Phase 1: Stabilize & Extend (Months 1-3)	13
5.2 Phase 2: Connect & Process (Months 4-6)	13

5.3 Phase 3: Migrate & ML (Months 7-9)	14
Chapter 6: Drone Survey Module Deep Dive	15
6.1 Flight Planning Math	15
6.2 Mission Export Formats	16
6.3 Live Drone Link (MAVSDK-Rust Sidecar)	17
6.4 In-app Photogrammetry (ODM Sidecar)	17
6.5 ML Feature Extraction (ONNX Runtime)	17
Chapter 7: Math Standards & Compliance	18
7.1 ASPRS 2014 (Positional Accuracy Standards for Remote Sensing)	18
7.2 NMAS 1947 (National Map Accuracy Standards)	19
7.3 ISO 19157 (Geographic Information - Data Quality)	19
7.4 RDM 1.1 / Cap. 299 (Kenya-Specific)	19
7.5 FGDC-STD-007.3 (NSSDA) and ICSM SP1 (Future Expansion)	20
Chapter 8: Engineering Practices (Agency-Agents Workflow)	21
8.1 The Twelve-Agent Core Roster	21
8.2 Sequential Handoff Workflow	22
8.3 Testing Methodology	22
8.4 Security Model	23
Chapter 9: Production Readiness & Release Plan	23
9.1 Code-Signing on Zero Budget	23
9.2 Auto-Update with Staged Rollout	24
9.3 Closed Beta Program (Volunteer-Based)	25
9.4 Bug Bounty Program (Community-Based)	25

9.5 Tutorial Videos (DIY) and Public Release	26
Chapter 10: Risk Matrix & Mitigation	26
10.1 Top Three Risks in Detail	27
Chapter 11: Budget, Timeline & KPIs	28
11.1 Budget Breakdown (Zero-Cost Path)	28
11.2 Trade-offs of the Zero-Cost Path	29
11.3 Timeline (Gantt-Style)	30
11.4 KPI Dashboard	30
Chapter 12: Conclusion & Next Actions	31
12.1 Top Five Next Actions (Next 30 Days, Zero Cost)	32

Chapter 1: Executive Summary

MetaRDU Desktop v1.0 is a credible, well-architected surveyor's office tool that consumes drone photogrammetry outputs and produces statutory survey deliverables for the Kenyan market. However, the product has a critical identity gap that this upgrade plan resolves. Despite its name suggesting drone mission planning, the current codebase contains zero flight planning math, zero camera footprint calculations, zero waypoint generation, zero MAVLink or MAVSDK integration, and zero in-app photogrammetry processing. The drone-related functionality that does exist is confined to Ground Control Point (GCP) management, a pipeline orchestrator that delegates all heavy lifting to external tools, and placeholder implementations for contour generation and feature extraction that return synthetic data. This plan transforms MetaRDU Desktop into a true production-grade drone survey workstation over a nine-month, three-phase upgrade.

The upgrade strategy is sequenced to deliver value continuously rather than as a single big-bang release. Phase 1 (Months 1-3, codenamed Stabilize & Extend) hardens the existing Electron shell with zod-validated IPC contracts, wires the five P0 math features into the UI, adds a complete flight planning engine with camera footprint math and lawnmower waypoint generation, integrates GDAL bindings to replace placeholder contour and feature extraction code, and stands up code-signing infrastructure plus a bug bounty program. Phase 2 (Months 4-6, codenamed Connect & Process) introduces the MAVSDK-Rust sidecar for live drone telemetry and mission upload, bundles OpenDroneMap as a local photogrammetry sidecar so processing works offline, and launches a closed beta program with five or more real Kenyan surveyors. Phase 3 (Months 7-9, codenamed Migrate & ML) executes the Tauri 2.x migration, adds ONNX-based machine learning feature extraction for building and road footprint delineation, produces three tutorial videos, and ships the public v2.0 release.

The scope spans five drone capabilities (flight planning, live drone link, in-app photogrammetry, real raster I/O, and ML feature extraction), five drone platform families (DJI, ArduPilot/PX4, Autel EVO, senseFly eBee, and generic KML/KMZ), and six math standards (ASPRS 2014, NMAS 1947, ISO 19157, RDM 1.1 / Cap. 299, FGDC-STD-007.3, and ICSM SP1). The architecture decision to migrate from Electron to Tauri 2.x is gated to Phase 3, after the Rust sidecar pattern has been proven on the existing Electron shell. This sequencing minimizes migration risk by validating the Rust integration story on a known-good foundation before committing to the full shell rewrite. The binary is expected to shrink from approximately 150 megabytes to under 15 megabytes, idle memory from 350 megabytes to under 120 megabytes, and cold start from 2.4 seconds to under 1.2 seconds.

Headline Metrics

Binary size 150 MB to 10 MB. Drone capabilities 0 to 5 (flight planning, live drone link, in-app photogrammetry, real raster I/O, ML feature extraction). Math standards 5 to 6 (adds ISO 19157). Production readiness gates 0 to 3 (midpoint Reality Check, pre-launch Reality Check, closed beta sign-off). Total budget: \$0 to \$109 USD per year (zero-cost path using SignPath Foundation, GitHub Releases, volunteer beta, and community bug bounty; optional \$99 Apple Developer ID when revenue allows).

The engineering methodology borrows the twelve-agent roster and sequential-handoff workflow from the msitarzewski/agency-agents repository. Each phase activates a specific subset of agents: Phase 0 begins with

the Codebase Onboarding Engineer producing a factual map of the current architecture; Phase 1 adds the Software Architect, Desktop App Engineer, Spatial Data Engineer, and Drone/Reality Mapping agents; Phase 2 adds the Web GIS Developer, 3D & Scene Developer, and GIS QA Engineer; Phase 3 adds the Test Automation Engineer, Security Architect, Application Security Engineer, and Reality Checker. The Reality Checker runs twice as a quality gate: once at the midpoint of Phase 2 to decide whether to proceed to Phase 3 or pivot, and once before the public v2.0 release as a GO/NO-GO decision requiring evidence per criterion.

Risks are significant but manageable. The Tauri migration is the largest single risk because it requires re-testing all 118 IPC handlers and replacing the native better-sqlite3 binding with rusqlite. The mitigation is to execute the migration only after the Rust sidecar pattern is proven in Phase 1 and 2, and to maintain a parallel Electron release branch throughout Phase 3 so the team can roll back if the Tauri build fails to stabilize. The OpenDroneMap sidecar is the second largest risk because it adds a Docker or PyInstaller dependency that may be too heavy for low-end field laptops. The mitigation is to make photogrammetry an optional feature with a clear system-requirements check, and to keep the external WebODM server integration as a fallback for high-end users.

Chapter 2: Current State Assessment

MetaRDU Desktop v1.0.0 was tagged in July 2026 as the first desktop release of the metardu codebase, which originally shipped as a Next.js browser application. The desktop rewrite was motivated by severe performance bottlenecks in the browser version, including repeated out-of-memory failures during Next.js production builds on 4 GB sandboxes, server-side compute offload via BullMQ and Redis that added latency to every correction call, an absolute dependency on an external WebODM server for any drone photogrammetry work, and a sprawling operational stack of Docker, Caddy, nginx, Cloudflare Tunnel, PM2, and a Python FastAPI worker. The desktop rewrite collapses this entire stack into a single Electron binary backed by SQLite with Spatialite, eliminating the network round-trip for every compute operation and enabling true offline field work.

2.1 Tech Stack

The current stack is Electron 31 wrapping a React 18 renderer with OpenLayers 10 for 2D map rendering and three.js for 3D point cloud visualization. The backend is the Electron main process running Node.js 20, with better-sqlite3 providing synchronous access to a local SQLite database extended with Spatialite for spatial indexing. The compute engine lives in a separate workspace package, @metardu/engine, which is pure TypeScript and reuses approximately 95% of the upstream metardu web codebase verbatim. Inter-process communication uses Electron's contextBridge with 118 registered IPC handlers across 25 namespaces, every one wrapped with rate-limiting at 120 calls per minute per channel, idempotency for concurrent writes, structured error envelopes, and observability metrics for slow-computation detection.

The codebase is substantial: 490 TypeScript source files, approximately 145,971 lines of code, 86 test files with 1,259 engine tests passing at 100% and 75% branch coverage, and 31 acceptance and smoke scripts. Five Architecture Decision Records (ADRs 001 through 005) document the major design choices. The CI pipeline runs on GitHub Actions across a three-OS matrix of Ubuntu, Windows, and macOS with fail-fast disabled.

Packaging produces Windows NSIS and MSIX installers, macOS DMG for x64 and arm64, and Linux .deb and AppImage. Auto-update is configured via electron-updater pointing at GitHub Releases, though code-signing certificates have not yet been purchased and the auto-update channel has never been verified against a real minor version bump.

2.2 Three Surveying Verticals

The product covers three surveying verticals that map directly to the Kenyan regulatory framework. Cadastral surveying implements Bowditch and Transit traverse closure with precision evaluation per Survey Act Cap. 299, generates Form No. 4 deed plan PDFs in A1 through A4 sizes, exports NLIMS and ArdhiSasa JSON with schema validation, produces a nine-sheet statutory computation workbook in Excel, generates mutation forms for subdivision and amalgamation, and applies an RSA-2048 cryptographic surveyor certificate seal to every deliverable. Topographic surveying implements breakline-aware constrained Delaunay TIN claiming 50,000 points in 100 milliseconds, marching-triangle contours at any interval, RINEX 2/3/4 import, LAS/LAZ point cloud import, DXF export with a 61-layer Survey of Kenya registry, LandXML 1.2/GeoJSON/Shapefile export, a feature coding library of 70 codes across 10 categories, and a GIS QA Report with PASS, CONDITIONAL, or FAIL verdicts.

Engineering surveying implements horizontal and vertical curve design including clothoid spiral transitions, superelevation computation, leveling by rise and fall plus height of collimation with 10 times the square root of K millimeter closure per RDM 1.1, cross-section volume by both prismoidal and end-area methods, mass-haul diagram optimization, AASHTO pavement design with ESA and layer structure, slope analysis via IDW interpolation and classification, staking table generation, road reserve compliance checking, as-built survey comparison, and machine-control export in seven formats including LandXML, DXF, Trimble, Leica, Topcon, generic, and stakeout CSV. The math engine is genuinely strong and well-cited to standard surveying literature including Snyder USGS Professional Paper 1395, Ghilani and Wolf Adjustment Computations, Basak, Schofield and Breach, AASHTO, and RDM 1.3.

2.3 Drone Capabilities: Present vs Absent

The drone-related code in v1.0 is the weakest part of the product, despite the metardu brand suggesting a drone focus. A full ripgrep of the codebase for MAVLink, MAVSDK, DroneKit, and related identifiers returned zero matches, confirming that no drone protocol integration exists. The drone-imagery.ts module is explicitly documented as a downstream consumer: METARDU does not process drone photos, that is OpenDroneMap or Pix4D's job, and instead METARDU takes the outputs of those tools and turns them into statutory survey deliverables. The aerial-pipeline.ts orchestrator models a 13-stage pipeline but stages 3 (drone capture) and 4 (photogrammetry processing) are marked external and not executed by METARDU. The contour generation function returns synthetic concentric circles, and the feature extraction function returns 10 hardcoded square building footprints and one horizontal road line.

Strength	Status	Gap	Status
Cassini-Soldner projection (1,922 LOC, exact to D6)	Full	No MAVLink/MAVSDK/DroneKit integration	Missing

Strength	Status	Gap	Status
Helmert datum transform (Gauss-Newton, 726 LOC)	Full	No flight planning or camera footprint math	Missing
Least Squares Adjustment (3,560 LOC, parametric+robust+network)	Full	No waypoint generation or mission export	Missing
Clothoid spiral (522 LOC, full set-out)	Full	No in-app photogrammetry (delegated to ODM)	Missing
TIN with breaklines + marching-triangle contours	Full	No GDAL bindings (contour gen returns synthetic data)	Placeholder
GCP manager (4 export formats, residual verification)	Full	Feature extraction returns hardcoded squares	Placeholder
Aerial pipeline orchestrator (13 stages)	Full	118 IPC handlers use any type, no zod validation	Weak
RSA-2048 cryptographic surveyor seal	Full	5 P0 math features not wired to UI	Gap
1259/1259 engine tests, 75% branch coverage	Full	No Playwright E2E tests	Missing
3-OS CI matrix on GitHub Actions	Full	Code-signing certs not purchased	Blocked

2.4 Critical Gaps Summary

The single most misleading gap is the drone-imagery module. Its function signatures look production-ready but the implementations return synthetic data without warning the user. A surveyor who imports a real Digital Surface Model and invokes `generateContoursFromDSM` will receive concentric circle polygons that look plausible on a map but have no relationship to the actual terrain. This is worse than a missing feature because it produces confidently wrong output. The fix in Phase 1 is to integrate GDAL bindings via the `gdal-async` Node package or by shelling out to `gdal_contour`, `gdal_calc.py`, and `gdal_rasterize` as child processes, and to replace the synthetic return values with real raster I/O. Until this lands, the contour and feature extraction functions are explicitly marked as not safe for production use.

The second critical gap is the IPC validation surface. All 118 handlers accept input typed as `any`, which means malformed payloads from the renderer can crash the privileged main process or, worse, execute arbitrary file system operations. The zod validation library is already a dependency but is not used in the IPC layer. The fix in Phase 1 is to define one zod schema per IPC channel and to validate every input on the privileged side before any business logic runs. This is also a security gate because the renderer is, in the Desktop App Engineer agent's words, a browser tab with delusions of grandeur, and every IPC channel must be treated as a public API surface with the narrowest possible verb exposed.

The third critical gap is the absence of the five P0 math features from the UI. The engine has full implementations of Least Squares Adjustment, error ellipses with covariance propagation, Cassini to UTM bidirectional conversion, grid-to-ground distance correction, and clothoid transition curves, but none of these are exposed in the React components. The traverse panel only offers Bowditch, the road design UI lacks clothoid entry and exit spirals required by RDM 1.1 for design speeds above 50 kilometers per hour, and the

coordinate converter only handles one direction. The fix in Phase 1 is to wire each of these engine modules to a UI component, with property-based tests guarding the math against numerical regressions.

Chapter 3: Vision, Goals, and Success Metrics

The product vision for MetaRDU Desktop v2.0 is to be a production-grade, Tauri-based drone survey workstation that combines flight planning, live drone operations, in-app photogrammetry, and statutory survey deliverables for the Kenyan surveying industry, with a clear architectural path to pan-African and global expansion. The vision is deliberately narrow in scope (Kenya first, country packs later) and broad in capability (every step of the drone survey workflow from mission planning to sealed deed plan in one application). This dual focus on depth over breadth is what differentiates MetaRDU from general-purpose GIS tools like QGIS and from drone-specific tools like Pix4D or DroneDeploy, none of which produce statutory Kenyan survey deliverables.

3.1 Five Strategic Goals

Goal G1 is to migrate the desktop shell from Electron to Tauri 2.x with a Rust backend for heavy compute, shrinking the binary from approximately 150 megabytes to under 15 megabytes, reducing idle memory from 350 to under 120 megabytes, and enabling true native performance for TIN computation, point cloud processing, and MAVLink telemetry parsing. Goal G2 is to add five drone capabilities that are currently absent: flight planning with camera footprint and waypoint generation, live drone link via MAVSDK-Rust, in-app photogrammetry via an OpenDroneMap sidecar, real raster I/O via GDAL bindings, and ML feature extraction via ONNX Runtime. Goal G3 is to support five drone platform families: DJI Mavic, Mini, and Phantom via KMZ waypoint export; ArduPilot and PX4 via MAVLink mission upload; Autel EVO via KMZ; senseFly eBee via eMotion XML; and any platform via generic KML.

Goal G4 is to comply with six math standards: ASPRS Positional Accuracy Standards 2014 for remote sensing Class I, II, and III RMSE thresholds; NMAS 1947 legacy 90% circular error of 1/30 inch; ISO 19157 geographic information data quality elements; RDM 1.1 and Survey Act Cap. 299 for Kenya-specific compliance; FGDC-STD-007.3 NSSDA for future USA expansion; and ICSM SP1 for future Australia and New Zealand expansion. Goal G5 is to achieve full production readiness with code-signing certificates on all three platforms, a closed beta with at least 10 real Kenyan surveyors, a security bug bounty program, three tutorial videos, and a staged auto-update rollout with rollback capability.

3.2 Success Metrics

Each goal has three to five measurable Key Performance Indicators with target values and a defined measurement method. The table below summarizes the full success metrics dashboard that will be tracked weekly throughout the nine-month upgrade. Metrics in bold are gating criteria for the public v2.0 release: failure to meet any gating metric triggers a release block and a root-cause analysis at the weekly project review.

Goal	KPI	Target	Measurement
G1 Tauri	Binary size	< 15 MB	electron-builder / tauri build output size
G1 Tauri	Idle memory	< 120 MB	Activity Monitor / Task Manager after 5 min idle
G1 Tauri	Cold start	< 1.2 s	time-to-first-paint benchmark on M1 MacBook Air
G1 Tauri	IPC handler migration	118 / 118 re-tested	Playwright E2E coverage of every IPC channel
G2 Drone	Flight planning accuracy	GSD within 1% of spec	Property-based test vs known camera sensors
G2 Drone	MAVLink telemetry latency	< 200 ms p95	MAVSDK-Rust ping over USB serial
G2 Drone	Photogrammetry success rate	> 90%	ODM sidecar on 100-photo test dataset
G2 Drone	GDAL contour correctness	RMSE < 0.5 m vs survey	Compare generated contours to field-surveyed breaklines
G2 Drone	ML building footprint IoU	> 0.65	OpenCities AI test set evaluation
G3 Platforms	DJI KMZ validation	100% pass DJI Pilot	Manual upload to DJI Pilot 2 app
G3 Platforms	ArduPilot .waypoints	100% pass QGC	Mission Planner + QGroundControl import test
G4 Math	ASPRS Class I compliance	RMSE _{x,y} < 7.5 cm	GCP residual check on test dataset
G4 Math	ISO 19157 completeness	100% pass	GIS QA Engineer automated topology check
G4 Math	RDM 1.1 levelling closure	10 sqrt(K) mm	Property-based test on synthetic networks
G5 Release	Code-signing verification	SmartScreen + Gatekeeper pass	Manual download test on clean Win + macOS
G5 Release	Beta surveyor sign-off	>= 7 of 10 NPS > 8	Structured feedback form after 2-week trial
G5 Release	Auto-update success rate	> 99% over 1 minor bump	electron-updater / Tauri updater metrics
G5 Release	Crash-free sessions	>= 99.5%	Sentry crash reporting over 7-day window

The measurement cadence is weekly for engineering metrics (binary size, idle memory, IPC coverage, MAVLink latency, photogrammetry success rate), biweekly for product metrics (beta surveyor feedback, NPS, tutorial completion rates), and at every phase gate for release metrics (code-signing verification, auto-update success rate, crash-free sessions). The Reality Checker agent runs a full evidence audit at the Phase 2 midpoint and at the pre-launch gate, requiring screenshots, test outputs, and beta surveyor testimonials for every gating metric before issuing a GO verdict.

Chapter 4: Architecture Decisions (ADRs 006-012)

Seven new Architecture Decision Records govern the upgrade. Each follows the Status, Context, Decision, Consequences template established by ADRs 001 through 005 in the existing codebase. Every decision names what is given up, because there are no solutions, only trade-offs. The full ADR markdown files will be committed to the docs/adrs/ directory of the metardu-desktop repository as the first deliverable of Phase 1, and these summaries are the executive overview.

4.1 ADR-006: Rust Sidecar Strategy (Phase 1-2)

Status: Accepted. **Context:** Heavy compute operations including TIN generation, point cloud octree traversal, GDAL raster math, MAVLink message parsing, and OpenSfM feature matching are CPU-bound and benefit from native code. The current TypeScript implementations work but are 5 to 20 times slower than equivalent Rust code on benchmarks. A full Tauri migration in Phase 1 would be high-risk because all 118 IPC handlers would need re-testing simultaneously. **Decision:** Keep the Electron shell for Phases 1 and 2, but extract heavy compute into a Rust binary called via a length-prefixed JSON protocol over stdin and stdout. The sidecar is bundled as a platform-specific binary in the resources/ directory and spawned as a child process at app startup. **Consequences:** Adds one build target (Rust) to the toolchain, requires cross-compilation for Windows, macOS, and Linux, and introduces a process boundary that adds approximately 2 milliseconds of latency per call. The upside is that the Rust code is reusable when Tauri migration happens in Phase 3, making that migration substantially easier.

4.2 ADR-007: Tauri 2.x Migration (Phase 3)

Status: Accepted, deferred to Phase 3. **Context:** ADR-001 chose Electron for v1.0 because 95% of the upstream metardu TypeScript code could be reused verbatim, minimizing time to v1.0. The downside is a 150 megabyte binary, 350 megabyte idle memory, and a Chromium dependency that adds attack surface. Tauri 2.x uses the system WebView (WebView2 on Windows, WKWebView on macOS, WebKitGTK on Linux), shrinking the binary to under 15 megabytes and idle memory to under 120 megabytes. **Decision:** Migrate the shell to Tauri 2.x in Phase 3, after the Rust sidecar pattern is proven. Replace better-sqlite3 with rusqlite, replace electron-updater with tauri-plugin-updater, and replace electron-builder with tauri-cli bundling. The @metardu/engine TypeScript package is reused verbatim because it is framework-agnostic. **Consequences:** Two-month migration timeline, re-testing all 118 IPC handlers as Tauri commands, and a parallel Electron release branch must be maintained as a rollback path. The upside is a 10x smaller binary, native performance, and a significantly smaller attack surface.

4.3 ADR-008: Drone Protocol Stack

Status: Accepted. **Context:** Five drone platforms must be supported. DJI dominates the consumer and prosumer market with Mavic, Mini, and Phantom lines. ArduPilot and PX4 dominate the open-source autopilot market on Pixhawk hardware. Autel EVO is a smaller but growing DJI alternative. senseFly eBee is

the leading fixed-wing survey drone. Generic KML/KMZ covers any other platform. **Decision:** Implement mission export in five formats: DJI KMZ using the wpml XML schema consumed by DJI Pilot 2 and Litchi; ArduPilot .waypoints using the QGC WPL 110 text format consumed by Mission Planner and QGroundControl; Autel EVO KMZ (subset of DJI format); senseFly eMotion XML; and generic KML using the KML 2.2 Point geometry with ExtendedData for altitude and speed. Live telemetry uses MAVSDK-Rust for ArduPilot and PX4 only; DJI live telemetry requires the DJI Cloud API which is out of scope for v2.0 and deferred to v2.1. **Consequences:** Five export code paths to maintain, but each is well-documented by the respective vendor. The MAVSDK-Rust sidecar is the only live telemetry channel in v2.0.

4.4 ADR-009: Photogrammetry Sidecar (OpenDroneMap)

Status: Accepted. **Context:** The original metardu web app delegated all photogrammetry to an external WebODM server, which silently no-ops if the WEBODM_URL environment variable is unset. This is a hard ceiling for field work in Kenya where internet connectivity is unreliable. OpenDroneMap is the leading open-source photogrammetry engine and can run locally. **Decision:** Bundle ODM as a local sidecar in Phase 2. The preferred deployment is a Docker container spawned on-demand by the desktop app, with a fallback to a PyInstaller-bundled native ODM binary for systems without Docker. The existing 13-stage aerial pipeline orchestrator is preserved and updated to invoke the local ODM sidecar instead of an external server. **Consequences:** Adds a Docker or 2-gigabyte native binary dependency. Minimum system requirement is raised to 16 GB RAM and 50 GB free disk for photogrammetry work. The feature is gated behind a system-requirements check and can be skipped on low-end machines, with a clear external-WebODM fallback maintained.

4.5 ADR-010: GDAL Integration

Status: Accepted. **Context:** The current contour generation and feature extraction functions are placeholders that return synthetic data. Real raster I/O requires GDAL, the de facto standard open-source geospatial library. The Node.js ecosystem has two options: gdal-async provides native Node bindings, or shell out to gdal_contour, gdal_calc.py, and gdal_rasterize as child processes. **Decision:** Use gdal-async as the primary integration in Phase 1, with a shell-out fallback for operations not covered by the bindings (specifically gdal_contour and gdal_rasterize). GDAL is bundled as a platform-specific binary in resources/gdal/ to avoid the system dependency hell that has historically plagued GDAL deployments. **Consequences:** Adds approximately 80 megabytes to the binary per platform. The contour generation function in drone-imagery.ts is rewritten to call gdal_contour on real GeoTIFF inputs, and the synthetic concentric circle return is removed entirely. Feature extraction remains a placeholder until ML integration in Phase 3.

4.6 ADR-011: ML Pipeline (ONNX Runtime)

Status: Accepted, Phase 3. **Context:** Building footprint extraction, road centerline extraction, and change detection from orthophotos are high-value features that differentiate MetaRDU from competitors. The leading approach is to use pre-trained models from the OpenCities AI initiative or the Microsoft Building Footprints project, run inference locally via ONNX Runtime, and post-process the output into GeoJSON polygons. **Decision:** Use ONNX Runtime as the inference engine with pre-trained U-Net or Mask R-CNN models. The Rust sidecar hosts the ONNX Runtime via the ort crate, accepting orthophoto tiles as input and returning

polygon coordinates as output. Models are bundled in resources/models/ and are approximately 200 megabytes per model. **Consequences:** Adds 200 megabytes per model to the binary, but only the building footprint model is bundled by default; road and change-detection models are downloadable on demand. Inference latency is approximately 5 seconds per 512 by 512 pixel tile on an M1 MacBook Air, which is acceptable for batch processing but too slow for real-time use.

4.7 ADR-012: IPC Validation (Zod Schemas)

Status: Accepted, Phase 1 P0. **Context:** All 118 IPC handlers in v1.0 accept input typed as any, which is a critical security and reliability gap. The zod validation library is already a dependency but is not used in the IPC layer. Malformed payloads from the renderer can crash the privileged main process or execute unintended file system operations. **Decision:** Define one zod schema per IPC channel in a new packages/ipc-schemas/ workspace. Every handler validates input against its schema before any business logic runs, and returns a structured 400 error on validation failure. The preload.ts file is regenerated from the schemas to give the renderer end-to-end type safety. **Consequences:** Adds approximately 2 milliseconds of validation overhead per IPC call, which is negligible. The renderer-to-main contract becomes the single source of truth, eliminating an entire class of runtime type errors. The security posture improves because every privileged operation is now gated by an explicit allowlist of inputs.

Chapter 5: Phased Roadmap (3 Phases × 3 Months)

The upgrade is sequenced as three phases of three months each, with each phase shipping a tagged release to the closed beta channel. The sequencing prioritizes risk reduction: the highest-risk architectural change (Tauri migration) is deferred to Phase 3, after the Rust sidecar pattern has been proven on the existing Electron shell in Phases 1 and 2. The lowest-risk, highest-value features (flight planning, GDAL, zod validation) ship first in Phase 1 to demonstrate progress to stakeholders and to give the closed beta surveyors something concrete to test in Phase 2. The full Gantt-style timeline is summarized in the table below, followed by detailed deliverables, milestones, exit criteria, and the active agent roster for each phase.

Work Stream	M1	M2	M3	M4	M5	M6	M7	M8	M9
zod IPC validation	X	X							
P0 math features in UI	X	X							
Flight planning engine		X	X						
GDAL bindings		X	X						
Code-signing setup			X						
MAVSDK-Rust sidecar				X	X				
ODM photogrammetry sidecar				X	X	X			

Work Stream	M1	M2	M3	M4	M5	M6	M7	M8	M9
Closed beta program					X	X			
Tauri 2.x migration							X	X	X
ML feature extraction								X	X
Tutorial videos								X	X
Public v2.0 release									X

5.1 Phase 1: Stabilize & Extend (Months 1-3)

Phase 1 hardens the existing Electron shell and adds the highest-value missing features. The deliverables are: a packages/ipc-schemas/ workspace with zod schemas for all 118 IPC handlers, validated and tested; the five P0 math features wired into the UI (LSA in traverse panel, error ellipses on adjusted coordinates, Cassini to UTM bidirectional converter, grid-to-ground distance correction, clothoid transition curves in road design); a new flight planning module in @metardu/engine with camera footprint math, GSD calculation, lawnmower waypoint generation, terrain-aware flight height from DTM, and mission export in five formats (DJI KMZ, ArduPilot .waypoints, MAVLink, Litchi CSV, senseFly eMotion XML); GDAL bindings via gdal-async with the contour generation function rewritten to use real gdal_contour; code-signing certificates purchased and configured for Windows EV and Apple Developer ID with notarization; and a bug bounty program launched on GitHub Security Advisories.

The milestone cadence is monthly. Month 1 ends with the zod schemas complete and the first two P0 math features shipped. Month 2 ends with all five P0 math features shipped, the flight planning engine complete with camera footprint and waypoint generation, and GDAL bindings integrated. Month 3 ends with the five mission export formats complete, code-signing verified on Windows and macOS, and the bug bounty program live. The exit criteria for Phase 1 are: 100% IPC handler zod coverage with property-based tests passing, all five P0 math features accepted by a GIS QA Engineer review, flight planning produces correct GSD within 1% of spec on a test camera sensor database, GDAL contour generation matches field-surveyed breaklines within 0.5 meters RMSE, and code-signed builds pass SmartScreen and Gatekeeper on clean installations.

The active agent roster for Phase 1 is: Codebase Onboarding Engineer (Month 1 only, produces the factual architecture map), Software Architect (full phase, owns ADRs 006 through 012), Desktop App Engineer (full phase, owns zod IPC validation and code-signing), Spatial Data Engineer (Months 2-3, owns GDAL integration), Drone/Reality Mapping agent (Months 2-3, owns flight planning math and mission export), and GIS QA Engineer (Month 3 only, runs the math and GDAL acceptance gates). The Reality Checker is not active in Phase 1 because no release ships to external users; the phase gate is a Code Reviewer agent review of every PR.

5.2 Phase 2: Connect & Process (Months 4-6)

Phase 2 adds live drone connectivity and in-app photogrammetry, and launches the closed beta. The deliverables are: a MAVSDK-Rust sidecar binary that connects to ArduPilot and PX4 autopilots over USB

serial or UDP, streaming live telemetry including HEARTBEAT, ATTITUDE, GPS_RAW_INT, and BATTERY_STATUS messages; a Tauri-compatible mission upload pipeline that converts the internal mission representation to MAVLink MAV_CMD_NAV_WAYPOINT messages and uploads them to the connected drone; an OpenDroneMap sidecar bundled as a Docker container with a PyInstaller fallback for systems without Docker, integrated into the existing 13-stage aerial pipeline; the aerial pipeline orchestrator updated to invoke the local ODM sidecar instead of an external WebODM server; and a closed beta program with five or more real Kenyan surveyors, structured feedback forms, and weekly office hours.

The exit criteria for Phase 2 are: MAVLink telemetry latency under 200 milliseconds p95 over USB serial on a Pixhawk 4 test drone; mission upload succeeds on at least three different ArduPilot and PX4 firmware versions; ODM sidecar completes a 100-photo test dataset with a success rate above 90%; the closed beta program has at least seven of ten surveyors reporting a Net Promoter Score above 8 after a two-week trial; and the Reality Checker midpoint gate issues a GO verdict with evidence per criterion. If the Reality Checker issues a NO-GO at midpoint, the team pivots to addressing the flagged issues before proceeding to Phase 3.

The active agent roster for Phase 2 adds: Web GIS Developer (owns the live telemetry dashboard UI), 3D & Scene Developer (owns the ODM point cloud viewer integration), and the Reality Checker (runs the midpoint gate at end of Month 6). The Drone/Reality Mapping agent continues from Phase 1 and owns the MAVSDK-Rust sidecar. The Test Automation Engineer joins in Month 6 to stand up the Playwright E2E suite for the new live drone link and photogrammetry journeys, using deterministic mock MAVLink and ODM responses to avoid flakiness.

5.3 Phase 3: Migrate & ML (Months 7-9)

Phase 3 executes the Tauri migration, adds ML feature extraction, and ships the public v2.0 release. The deliverables are: a Tauri 2.x shell with all 118 IPC handlers migrated from Electron contextBridge to Tauri commands, with the @metardu/engine TypeScript package reused verbatim and the better-sqlite3 binding replaced with rusqlite; the Rust sidecar from Phase 1 and 2 absorbed into the main Tauri binary because Tauri is already Rust; an ONNX Runtime ML pipeline for building footprint extraction using a pre-trained U-Net model, with the Rust ort crate hosting inference and returning GeoJSON polygons; road centerline extraction using a separate pre-trained model; change detection between two orthophoto epochs; three tutorial videos of approximately 10 minutes each covering the cadastral, topographic, and drone survey workflows; and the public v2.0.0 release tagged on GitHub with auto-generated changelog, download page, and social announcement.

The exit criteria for Phase 3 are: binary size under 15 megabytes on all three platforms; idle memory under 120 megabytes; cold start under 1.2 seconds on an M1 MacBook Air; all 118 IPC handlers pass the Playwright E2E suite on Tauri; ML building footprint extraction achieves an Intersection over Union above 0.65 on the OpenCities AI test set; the closed beta surveyors from Phase 2 sign off on the Tauri build after a one-week trial; the Reality Checker pre-launch gate issues a GO verdict; and the first staged rollout at 1% adoption maintains a crash-free session rate above 99.5% over a 7-day window before expanding to 10% and then 100%.

The active agent roster for Phase 3 adds: the Security Architect and Application Security Engineer (run the threat model on the new Tauri IPC boundary and the ML inference pipeline), the Test Automation Engineer

(full phase, owns the Tauri migration E2E suite), the DevOps Automator (owns the staged rollout and the auto-update verification), the Performance Benchmarker (verifies the footprint budgets), the Technical Writer (owns the tutorial video scripts and the v2.0 release notes), and the Reality Checker (runs the pre-launch GO/NO-GO gate). The SRE agent is on standby for the first 72 hours after the public v2.0 release to handle any incidents.

Chapter 6: Drone Survey Module Deep Dive

This chapter is the technical specification for the five drone capabilities added across Phases 1, 2, and 3. Each section covers the math, the data flow, the file formats, and the integration points with the existing @metardu/engine and the Electron or Tauri shell. The math is presented with explicit formulas so that the GIS QA Engineer can verify correctness against standard references including Colomina and Molina 2014, ASPRS 2014, and the relevant ISO standards.

6.1 Flight Planning Math

Flight planning computes the mission parameters required to achieve a target Ground Sample Distance (GSD) and image overlap, given a camera sensor specification and a survey area polygon. The fundamental relationship is $GSD = \frac{\text{sensor pixel size} \times \text{altitude}}{\text{focal length}}$. Equivalently, $GSD = \frac{\text{sensor width} \times \text{altitude}}{\text{image width} \times \text{focal length}}$. The camera footprint on the ground is the image footprint width equals the altitude multiplied by the sensor width divided by the focal length, and the image footprint height equals the altitude multiplied by the sensor height divided by the focal length. These two formulas are the foundation of every other flight planning calculation.

The lawnmower waypoint generation algorithm takes the survey area polygon, the target GSD, the front overlap percentage, the side overlap percentage, and the camera sensor specification, and produces a list of waypoints. The line spacing is the image footprint height multiplied by one minus the side overlap, expressed as a decimal. The photo spacing along a flight line is the image footprint width multiplied by one minus the front overlap. The flight lines are oriented along the longest dimension of the survey area bounding box to minimize the number of turns, which are the most battery-expensive maneuver. Terrain-aware flight height adjusts the altitude above ground at each waypoint using a Digital Terrain Model, so that the GSD remains constant over rolling terrain. This requires a DTM raster input and adds approximately 15% to the flight time compared to a flat-terrain flight.

Battery and flight time estimation uses the drone's published cruise speed, the published battery capacity, and a 20% safety margin. The total flight time is the sum of the straight-line distance divided by cruise speed plus 10 seconds per turn for deceleration, rotation, and acceleration. The number of batteries required is the total flight time divided by the usable battery time, rounded up. The usable battery time is the published flight time at the cruise speed and the mission altitude, multiplied by 0.8 for the safety margin. For a typical DJI Mavic 3 Enterprise at 75 meters altitude, 8 meters per second cruise speed, 75% front overlap, and 65% side overlap, a 50-hectare survey requires approximately 18 minutes of flight time and one battery.

Parameter	Symbol	Formula	Example (Mavic 3 Enterprise)
Sensor width	Sw	Spec	17.9 mm
Sensor height	Sh	Spec	13.0 mm
Image width	Iw	Spec	5280 px
Image height	Ih	Spec	3956 px
Focal length	f	Spec	12.0 mm
Altitude AGL	H	User input	75 m
Pixel size	ps	Sw / Iw	0.00339 mm
GSD	g	$ps \times H / f$	2.12 cm/px
Footprint width	Fw	$H \times Sw / f$	111.9 m
Footprint height	Fh	$H \times Sh / f$	81.3 m
Front overlap	P_front	User input	75%
Side overlap	P_side	User input	65%
Photo spacing	d_photo	$Fw \times (1 - P_front)$	28.0 m
Line spacing	d_line	$Fh \times (1 - P_side)$	28.5 m

6.2 Mission Export Formats

Five mission export formats are implemented in Phase 1. The DJI KMZ format uses the wpml XML schema consumed by DJI Pilot 2 and Litchi. The KMZ is a ZIP archive containing a mission.kml file with the waypoint coordinates, altitudes, speeds, and actions, plus a template.kml file with the mission metadata. The ArduPilot .waypoints format is the QGC WPL 110 text format, with one line per waypoint containing the index, coordinate type (0 for MAV_CMD_NAV_WAYPOINT, 16 for MAV_CMD_NAV_RETURN_TO_LAUNCH), latitude, longitude, altitude, and parameters. This format is consumed by Mission Planner and QGroundControl. The MAVLink mission upload uses the MAVSDK-Rust sidecar to send MISSION_ITEM messages to the connected drone, with each waypoint as a separate MISSION_ITEM containing the target coordinate, altitude, and MAV_CMD_NAV_WAYPOINT command.

The Litchi CSV format is a simple comma-separated file with columns for latitude, longitude, altitude (relative to ground), gimbal pitch, gimbal yaw, and photo action. The senseFly eMotion XML format is a proprietary XML schema with a Mission element containing Waypoint children, each with Position (latitude, longitude, altitude Above Sea Level), PhotoAction, and TriggerDistance elements. The generic KML format uses the KML 2.2 specification with Point geometry for each waypoint and ExtendedData elements for altitude, speed, and action metadata, allowing import into any GIS application. Each export format is tested by round-tripping: generate a mission, export, re-import, and verify the waypoints match within 1 centimeter of horizontal error and 0.1 meters of vertical error.

6.3 Live Drone Link (MAVSDK-Rust Sidecar)

The live drone link uses a Rust sidecar built on the MAVSDK-Rust crate, which provides a high-level API over the MAVLink binary protocol. The sidecar connects to ArduPilot and PX4 autopilots over USB serial (typically `/dev/ttyACM0` on Linux, `COM3` on Windows) or UDP (typically 14550 for telemetry radios). The sidecar exposes a JSON-over-stdin/stdout protocol to the Electron or Tauri main process, with messages for connect, disconnect, start_telemetry_stream, upload_mission, start_mission, and rtl (Return to Launch). The telemetry stream is pushed from the sidecar to the main process at 5 Hz for HEARTBEAT and BATTERY_STATUS, and at 10 Hz for ATTITUDE and GPS_RAW_INT, matching the typical autopilot broadcast rates.

The React renderer subscribes to the telemetry stream via an IPC event listener and updates a live dashboard with the drone's position, altitude, battery percentage, voltage, current, heading, GPS fix type, satellite count, and flight mode. The mission upload pipeline serializes the internal mission representation to MAVLink MISSION_ITEM messages, sends them to the drone via the MAVSDK mission.upload_mission API, and verifies the upload by reading back the mission count. The start_mission command arms the drone (with a confirmation dialog in the UI for safety) and triggers the MAV_CMD_MISSION_START command. The RTL command sends the MAV_CMD_NAV_RETURN_TO_LAUNCH command, which initiates an automatic return to the launch point at the configured RTL altitude. All commands that affect drone state require a two-step confirmation in the UI: a click to arm, followed by a 5-second hold-to-confirm button, to prevent accidental activation.

6.4 In-app Photogrammetry (ODM Sidecar)

The in-app photogrammetry replaces the external WebODM server dependency with a local OpenDroneMap sidecar. The preferred deployment is a Docker container spawned on-demand by the desktop app, using the official opendronemap/odm image. The desktop app mounts the photo directory as a volume, writes a JSON configuration file with the processing options (orthophoto-resolution, dem-resolution, dsm, dtm, contour-resolution), and starts the container. Progress is streamed back via Docker's log API and displayed in the UI as a progress bar with the current stage name. On completion, the orthophoto, DSM, DTM, point cloud, and contour files are imported into the MetaRDU dataset registry using the existing drone-imagery.ts import pipeline.

For systems without Docker, a PyInstaller-bundled native ODM binary is provided as a fallback. This binary is approximately 2 gigabytes in size and includes a bundled Python runtime, the ODM Python code, and all required native dependencies (OpenSfM, OpenMVS, PDAL, GDAL, etc.). The binary is downloadable from the MetaRDU website as an optional component during installation. The minimum system requirement for photogrammetry work is 16 GB of RAM and 50 GB of free disk space, and a system-requirements check at startup gates the feature. The existing external WebODM server integration is preserved as a third option for high-end users who prefer cloud processing, so the user has three choices: local Docker, local native, or external server.

6.5 ML Feature Extraction (ONNX Runtime)

The ML feature extraction pipeline runs in Phase 3 and uses the ONNX Runtime via the Rust ort crate, hosted in the Tauri main process. Three models are supported: building footprint extraction using a U-Net model pre-trained on the OpenCities AI dataset, road centerline extraction using a separate U-Net model trained on the SpaceNet 2 dataset, and change detection between two orthophoto epochs using a Siamese network. The default installation bundles only the building footprint model (approximately 200 megabytes); the road and change-detection models are downloadable on demand from the MetaRDU website.

The inference pipeline accepts an orthophoto GeoTIFF as input, tiles it into 512 by 512 pixel chunks with a 64-pixel overlap to avoid boundary artifacts, runs each tile through the model, and stitches the predictions back together. Post-processing converts the predicted mask into GeoJSON polygons by tracing the mask contours using the OpenCV findContours algorithm, simplifying the polygons with the Douglas-Peucker algorithm, and filtering out polygons smaller than 10 square meters (noise) or larger than 10,000 square meters (likely errors). The output is a GeoJSON file with one polygon per building, ready for import into the MetaRDU parcel registry. Inference latency is approximately 5 seconds per 512 by 512 pixel tile on an M1 MacBook Air, so a 100-hectare survey at 5 centimeter per pixel GSD (approximately 200 million pixels, or 800 tiles) takes about 70 minutes of compute time, which is acceptable for batch processing but explicitly not real-time.

Chapter 7: Math Standards & Compliance

Six math standards govern the accuracy and quality of MetaRDU Desktop outputs. Each standard has a different scope, jurisdiction, and implementation status in the v1.0 codebase. The compliance matrix below maps each standard to its current implementation status and the gap that the upgrade plan closes. The standards are layered: ASPRS 2014 governs remote sensing and photogrammetry outputs (orthophotos, DSMs, point clouds); ISO 19157 governs the broader data quality of all geographic information; NMAS 1947 is a legacy standard still cited in some jurisdictions; RDM 1.1 and Cap. 299 are Kenya-specific and govern cadastral, engineering, and topographic surveys; FGDC-STD-007.3 is the US NSSDA standard for future USA expansion; and ICSM SP1 is the Australia and New Zealand standard for future ANZ expansion.

7.1 ASPRS 2014 (Positional Accuracy Standards for Remote Sensing)

The ASPRS Positional Accuracy Standards for Digital Geospatial Data (2014) define three accuracy classes for remote sensing products including orthophotos, digital surface models, and point clouds. Class I requires a horizontal RMSE of 7.5 centimeters (1:500 scale equivalent) and a vertical RMSE of 15 centimeters. Class II requires 15 centimeters horizontal and 30 centimeters vertical (1:1000 scale). Class III requires 37.5 centimeters horizontal and 75 centimeters vertical (1:2500 scale). The standard also defines the methodology: at least 20 check points distributed across the project area, with horizontal RMSE computed as the square root of the mean of the squared horizontal errors ($RMSE_r$ equals the square root of $RMSE_x$ squared plus $RMSE_y$ squared), and vertical RMSE computed as the square root of the mean of the squared vertical errors.

The v1.0 codebase already implements ASPRS Class I, II, and III thresholds in the drone accuracy checker (tools/drone/page.tsx in the upstream web app, ported to the desktop drone-imagery module). The check is one-directional: the surveyor inputs the residuals manually, and the checker computes the RMSE and assigns a

class. The Phase 1 upgrade parses the residuals directly from the ODM or Pix4D quality report XML, eliminating manual transcription. The Phase 2 upgrade adds the GIS QA Engineer agent as a gate that automatically runs the ASPRS check before any orthophoto or DSM is published to the dataset registry, blocking publication if the residual RMSE exceeds the user-specified class threshold.

7.2 NMAS 1947 (National Map Accuracy Standards)

The National Map Accuracy Standards (NMAS) of 1947 are a legacy US standard still cited in some jurisdictions, particularly for printed maps. The standard requires that 90% of well-defined points tested shall be within 1/30 inch (0.85 millimeters) of their true position on a map at the published scale. For a 1:1000 scale map, this corresponds to 0.85 meters on the ground; for 1:5000, 4.25 meters. Vertical accuracy requires 90% of elevations to be within half the contour interval. NMAS is largely superseded by the NSSDA (FGDC-STD-007.3) which uses RMSE instead of the 90% threshold, but NMAS remains a useful reference for historical comparisons and for clients who specifically request it.

The v1.0 codebase does not implement NMAS. The Phase 1 upgrade adds a NMAS compliance check alongside the existing ASPRS check, computing the equivalent NMAS accuracy for a given map scale and reporting both ASPRS class and NMAS 90% threshold in the accuracy report. This is a low-effort addition because the math is simply scale-dependent: $\text{NMSE_accuracy} = \frac{\text{map scale denominator} \times 0.85 \text{ millimeters}}{1000}$ to convert to meters. The implementation is approximately 50 lines of TypeScript in the drone-accuracy module.

7.3 ISO 19157 (Geographic Information - Data Quality)

ISO 19157:2013 establishes the principles for describing the quality of geographic data and specifies the data quality elements, sub-elements, data quality measures, and the identification mechanism. The standard defines five data quality elements: positional accuracy (further subdivided into absolute accuracy, relative accuracy, and gridded data position accuracy), thematic accuracy (classification correctness and non-quantitative attribute correctness), temporal accuracy (temporal validity and temporal accuracy), logical consistency (conceptual, domain, format, and topological consistency), and completeness (commission and omission). For each element, the standard provides a catalog of measures with their formulas and units.

The v1.0 codebase implements positional accuracy (via ASPRS) and logical consistency (via the GIS QA Report's topology checks), but does not implement thematic accuracy, temporal accuracy, or completeness measures. The Phase 2 upgrade adds the missing elements: thematic accuracy via the ML feature extraction model's classification confidence score; temporal accuracy via the dataset registry's capture timestamp and validity window; completeness via a comparison between the expected survey area polygon and the actual orthophoto coverage polygon, reporting any gaps as omission errors. The GIS QA Engineer agent runs all five elements as a gate before any dataset is published, producing an ISO 19157-compliant quality report embedded in the dataset metadata.

7.4 RDM 1.1 / Cap. 299 (Kenya-Specific)

The Kenya Roads Design Manual (RDM 1.1, 2025 edition) and the Survey Act Cap. 299 are the primary regulatory frameworks for surveying in Kenya. RDM 1.1 Section 8 specifies a levelling closure tolerance of 10

times the square root of K millimeters, where K is the levelling line length in kilometers. For a 1-kilometer line, the tolerance is 10 millimeters; for 4 kilometers, 20 millimeters; for 16 kilometers, 40 millimeters. The Survey Act Cap. 299 specifies an angular misclosure of 15 seconds times the square root of N, where N is the number of traverse stations, and a linear misclosure of 1:10000 for urban surveys and 1:5000 for rural surveys. These constants are already implemented in the v1.0 engine and are part of the Form No. 4 deed plan generation pipeline.

RDM 1.1 Section 5.2.4 also requires clothoid transition curves for all roads with a design speed above 50 kilometers per hour. The v1.0 engine has a full 522-line clothoid implementation (spiral parameter A equals the square root of R times Ls, spiral angle tau equals Ls divided by 2R, curve shift p, tangent offset q, TS/SC/CS/ST chainages, modified tangent Ts equals (R+p) times tan of half the intersection angle plus q), but this is not exposed in the road design UI. The Phase 1 upgrade wires the clothoid engine module to the road design React component, making it the default for design speeds above 50 kilometers per hour and producing the TS, SC, CS, and ST chainages in the setting-out sheet. This is one of the five P0 math features identified in the v1.0 math improvement plan.

7.5 FGDC-STD-007.3 (NSSDA) and ICSM SP1 (Future Expansion)

The FGDC-STD-007.3 standard, also known as the National Standard for Spatial Data Accuracy (NSSDA), is the US federal standard for reporting positional accuracy of geospatial data. It uses the RMSE methodology similar to ASPRS 2014 but reports accuracy as the 95% confidence interval, computed as 1.96 times the RMSE for horizontal accuracy (assuming normally distributed errors) and 1.96 times the RMSE for vertical accuracy. The ICSM SP1 standard (Special Publication 1, Intergovernmental Committee on Surveying and Mapping) is the equivalent Australian and New Zealand standard, with similar RMSE-based methodology but slightly different confidence interval factors (1.96 for horizontal at 95%, 1.96 for vertical at 95%).

Neither standard is implemented in v1.0 because the product is Kenya-focused. Both are added in Phase 3 as part of the country-pack architecture expansion, which introduces USA and Australia/New Zealand country packs with their respective CRS (WGS84 / ETRS89 / GDA2020), deed plan templates, and accuracy standards. The implementation is straightforward because the math is identical to ASPRS with a different reporting convention: NSSDA accuracy equals 1.96 times RMSE. The country-pack architecture (ADR-005) already supports per-country standards, so adding the USA and ANZ packs is approximately one week of work each, plus the regulatory research to identify the equivalent of Form No. 4 in each jurisdiction.

Standard	Scope	v1.0 Status	Phase	Action
ASPRS 2014	Remote sensing	Partial (manual input)	P1	Auto-parse ODM/Pix4D residuals
NMAS 1947	Printed maps	Missing	P1	Add scale-based 90% check
ISO 19157	Geographic data quality	Partial (positional + logical)	P2	Add thematic + temporal + completeness
RDM 1.1 / Cap. 299	Kenya (cadastral/eng/topo)	Engine complete, UI partial	P1	Wire 5 P0 math features to UI
FGDC-STD-007.3	USA (future)	Missing	P3	Add USA country pack

Standard	Scope	v1.0 Status	Phase	Action
ICSM SP1	ANZ (future)	Missing	P3	Add ANZ country pack

Chapter 8: Engineering Practices (Agency-Agents Workflow)

The msitarzewski/agency-agents repository provides 230-plus specialized AI agent personas, each with a distinct identity, mission, critical rules, technical deliverables, and success metrics. The framework is not a runtime system but a set of installable agent definitions that slot into Claude Code, Cursor, Copilot, and other AI-assisted development tools. For the MetaRDU Desktop upgrade, twelve agents from the Engineering, GIS, Testing, Security, and Project Management divisions form the core team, with five additional agents activated on-demand for specific phases. The workflow follows the sequential-handoff-with-quality-gates pattern documented in the agency-agents examples.

8.1 The Twelve-Agent Core Roster

The twelve agents are activated in a specific sequence that matches the four-phase workflow. Phase 0 activates the Codebase Onboarding Engineer in read-only mode to produce a factual map of the current architecture, the IPC surface, the dependencies, and the tech-debt inventory. This map is the input to every subsequent agent. Phase 1 adds the Software Architect (owns ADRs and the trade-off matrix), the Desktop App Engineer (owns the zod IPC validation, code-signing, and auto-update infrastructure), the Spatial Data Engineer (owns GDAL integration and CRS reprojection), the Drone/Reality Mapping agent (owns the flight planning math and mission export), and the GIS QA Engineer (runs the math and GDAL acceptance gates at the end of Phase 1).

Phase 2 adds the Web GIS Developer (owns the live telemetry dashboard UI), the 3D & Scene Developer (owns the ODM point cloud viewer integration), and the Reality Checker (runs the midpoint gate at the end of Month 6). The Test Automation Engineer joins in Month 6 to stand up the Playwright E2E suite for the new live drone link and photogrammetry journeys. Phase 3 adds the Security Architect and Application Security Engineer (run the threat model on the new Tauri IPC boundary and the ML inference pipeline), the Performance Benchmark (verifies the footprint budgets), the Technical Writer (owns the tutorial video scripts and the v2.0 release notes), and the Reality Checker again for the pre-launch GO/NO-GO gate.

#	Agent	Division	Phase	Primary Deliverable
1	Codebase Onboarding Engineer	Engineering	P0	Architecture map + tech-debt inventory
2	Software Architect	Engineering	P1-3	ADRs 006-012 + trade-off matrix + C4 diagrams
3	Desktop App Engineer	Engineering	P1-3	zod IPC + code-signing + Tauri migration
4	Spatial Data Engineer	GIS	P1-2	GDAL bindings + CRS reprojection + format conversion

#	Agent	Division	Phase	Primary Deliverable
5	Drone/Reality Mapping	GIS	P1-2	Flight planning + mission export + MAVSDK sidecar
6	GIS QA Engineer	GIS	P1-3	Topology/CRS/RMSE accuracy validation gates
7	Web GIS Developer	GIS	P2	Live telemetry dashboard + processing UI
8	3D & Scene Developer	GIS	P2	ODM point cloud viewer integration
9	Test Automation Engineer	Testing	P2-3	Deterministic Playwright E2E, sharded in CI
10	Security Architect	Security	P3	Threat model + SAST/DAST in CI
11	Application Security Engineer	Security	P3	Code-level vulnerability review
12	Reality Checker	Testing	P2,P3	Midpoint + pre-launch GO/NO-GO gates

8.2 Sequential Handoff Workflow

The agency-agents framework mandates sequential handoffs: each agent's output becomes the next agent's input, pasted in full because agents do not share memory. This is the opposite of the parallel-everything approach used by some multi-agent systems, and it exists because summarization loses critical context. For the MetaRDU upgrade, the handoff chain is: Codebase Onboarding Engineer produces the architecture map, which the Software Architect reads in full before writing the ADRs; the ADRs are read in full by the Desktop App Engineer before designing the zod schemas; the schemas are read in full by the Spatial Data Engineer before designing the GDAL integration; and so on. Every handoff is a git commit with a structured worklog entry in the project's worklog.md file.

Discovery-phase agents (Codebase Onboarding, Software Architect, Senior Project Manager, Sprint Prioritizer) run in parallel during Phase 0 and the first two weeks of Phase 1, because their outputs are independent. Build-phase agents (Desktop App Engineer, Spatial Data Engineer, Drone/Reality Mapping, Web GIS Developer, 3D & Scene Developer) run in parallel within each phase but with dependencies on the discovery outputs. Quality-phase agents (Test Automation Engineer, GIS QA Engineer, Security Architect, Reality Checker) run sequentially with explicit gates: the Test Automation Engineer's E2E suite must pass before the GIS QA Engineer runs the accuracy gates, and both must pass before the Reality Checker runs the GO/NO-GO gate.

8.3 Testing Methodology

The Test Automation Engineer agent enforces a strict determinism-over-coverage philosophy. The iron rule is that a flaky test is a bug with your name on it, and no hard sleeps are ever allowed. Tests wait on conditions (element state, network response, URL change) rather than fixed timeouts. Tests own their data via API-based factories, never via the UI, and each test worker gets isolated data with no shared seed users. The selector strategy is getByRole first, data-testid as an escape hatch, and never brittle CSS chains. The test pyramid

discipline pushes everything provable down to unit tests, with E2E reserved for journeys where the integration itself is the risk. The full E2E suite must run in under 10 minutes via sharding across CI workers.

Flaky tests leave the merge-blocking suite within 24 hours and enter a triage queue, never deleted without diagnosis. Every failure produces artifacts: trace, screenshot, video, console log, and network log. The phrase works on my machine is treated as a tooling failure, not an excuse. Numeric service level objectives are: pass rate at or above 99.5%, flake rate below 0.5%, retries trending to zero, and the full suite completing in under 10 minutes. The GIS QA Engineer agent adds domain-specific gates: topology validation (no self-intersections, no gaps, no overlaps), CRS consistency (every layer in the same CRS or explicitly reprojected), and RMSE accuracy assessment before any dataset is published to the registry.

8.4 Security Model

The Security Architect and Application Security Engineer agents enforce a secure-by-design, defense-in-depth posture. The trust boundary is the IPC layer: the renderer is treated as a browser tab with delusions of grandeur, with contextIsolation true, nodeIntegration false, and sandbox true (or the Tauri equivalent). Every IPC channel validates inputs on the privileged side using the zod schemas from ADR-012, and exposes the narrowest possible verb: saveUserExport(data) rather than writeFile(path, data). Remote content never gets privileges. The drone-data ingestion paths (KMZ import, ODM output import, MAVLink telemetry) are threat-modeled as untrusted inputs because they originate from external systems that may be compromised or buggy.

The security SDLC embeds SAST (Semgrep or CodeQL) and DAST (OWASP ZAP for the IPC surface) in the CI pipeline, with dependency scanning (Dependabot or Snyk) and secret scanning (GitHub Secret Scanning) running on every push. The Security Architect produces a threat model document at the start of Phase 3 covering the new Tauri IPC boundary, the MAVSDK-Rust sidecar, the ODM sidecar subprocess, the ONNX Runtime inference pipeline, and the auto-update channel. The threat model is reviewed by the Application Security Engineer who produces a code-level vulnerability report. The bug bounty program (Phase 1) accepts external reports on the same scope, with reward tiers of \$100 (low), \$500 (medium), \$2000 (high), and \$5000 (critical).

Chapter 9: Production Readiness & Release Plan

Production readiness on a zero-budget is not only possible, it is the path that most successful open-source desktop applications have taken. VS Code, Obsidian (in its early days), Blender, and countless Electron and Tauri apps shipped for years using free code-signing alternatives, community bug bounties, and volunteer beta testers. The principle is to trade money for time and community: every paid infrastructure component has a free equivalent that costs more in setup effort but produces an equally production-ready result. The Desktop App Engineer agent's critical rule still applies: the auto-updater is the most critical code you own, and it must be tested before launch. What changes is how we achieve trusted distribution, not whether we achieve it.

9.1 Code-Signing on Zero Budget

Code-signing is the hardest problem to solve for free, because the certificate authorities (DigiCert, SSL.com, Sectigo) charge real money for the trust they provide. However, three free or near-free paths exist, and the right choice depends on the platform. For Windows, the primary path is **SignPath Foundation**, a non-profit that provides free code-signing certificates to open-source projects. The application requires a public GitHub repository with an OSI-approved license (MetaRDU Desktop is MIT licensed, which qualifies), a maintained release history, and a clear description of the project. Approval takes 1-2 weeks. Once approved, SignPath integrates directly with GitHub Actions via their app, signing every release build automatically with a certificate that SmartScreen recognizes. This eliminates the Windows EV certificate cost entirely.

The fallback for Windows, if SignPath approval is delayed or rejected, is a **self-signed certificate** generated locally via PowerShell's `New-SelfSignedCertificate` cmdlet. The signed binary will trigger a SmartScreen warning ('Windows protected your PC') on first run, which users bypass by clicking 'More info' then 'Run anyway'. This is the same warning that hundreds of indie games and open-source tools live with, and it does not prevent installation. The mitigation is documentation: the metardu.com download page and the README must include screenshots showing exactly how to bypass SmartScreen, with clear language explaining that the warning appears because the app is not signed by a paid certificate authority, not because it is malware. Over time, as download volume grows, SmartScreen reputation builds organically and the warning disappears, typically after a few thousand downloads with a low report rate.

For macOS, the situation is different. Apple does not offer a free code-signing path for distributed applications, and there is no equivalent of SignPath for macOS. The realistic options are: **Option A** (recommended when budget allows) is the Apple Developer ID at \$99 per year, which is the only way to notarize the app and avoid the 'unidentified developer' Gatekeeper warning. **Option B** (zero-cost) is to skip notarization entirely and document the bypass: macOS users right-click the app, select 'Open', and confirm the dialog, or run `xattr -d com.apple.quarantine /Applications/MetaRDU.app` in Terminal. This is the path that many open-source macOS apps take, including Inkscape and Audacity in their early years. **Option C** is to distribute via Homebrew Cask, where the Homebrew community accepts unsigned apps with appropriate caveats. The plan assumes Option B for the initial v2.0 release, with Option A as the first priority once MetaRDU generates any revenue.

For Linux, code-signing is free and always has been. The AppImage is signed with a self-generated GPG key (`gpg --gen-key`), the public key is published to a keyserver, and the AppImage checksum is published alongside the download. The .deb package is signed with `dpkg-sig` using the same GPG key. Linux desktop users who care about verification can import the GPG key and verify the signature, but most users rely on the package manager's checksum verification. This is the standard Linux distribution model and requires zero budget.

Code-Signing Decision Summary

Windows: SignPath Foundation (FREE, 1-2 week approval) as primary; self-signed + SmartScreen bypass docs as fallback. macOS: skip notarization for v2.0 (FREE); document the right-click Open workaround; upgrade to Apple Developer ID (\$99/yr) as first priority once revenue exists. Linux: self-generated GPG key (FREE). Total code-signing cost for v2.0 launch: \$0.

9.2 Auto-Update with Staged Rollout

The auto-update infrastructure is free because it uses GitHub Releases as the update provider, which is free for public repositories. `electron-updater` (Phases 1-2) and `tauri-plugin-updater` (Phase 3) both support GitHub Releases natively, requiring only a GitHub personal access token with repo permissions stored as a CI secret. A new release is published by tagging a git commit, pushing the tag, and letting GitHub Actions build and upload the platform-specific binaries as release assets. The update manifest (`latest.yml` or `latest.json`) is generated by the build script and uploaded alongside the binaries. The auto-update check runs at app startup and every 4 hours, with a 15-minute jitter to avoid thundering herd on the GitHub Releases API.

The staged rollout follows the Desktop App Engineer's pattern: 1% adoption for the first 7 days, expanding to 10% if the crash-free session rate stays above 99.5% and the update-success rate stays above 95%, then expanding to 100% after another 7 days at 10% with the same gates. The rollout percentage is controlled by the update manifest's `stagingPercentage` field. If the crash-free rate drops below 99.5% at any stage, the rollout is paused and the previous manifest is republished, triggering an automatic rollback for users who have not yet updated. A rollback drill is run before the public v2.0 release: publish a v2.0.1-rc with a known bug, verify the crash detection via Sentry (free tier), republish v2.0.0 as latest, and verify that users on v2.0.1-rc roll back automatically. The entire auto-update infrastructure costs \$0.

9.3 Closed Beta Program (Volunteer-Based)

The closed beta program runs on volunteer participation rather than paid stipends. This is how most open-source beta programs operate, and it produces equally valuable feedback when the product solves a real problem for the testers. The recruitment targets two groups: **licensed surveyors from the Institution of Surveyors of Kenya (ISK) and the Engineers Board of Kenya (EBK)** who already use drone photogrammetry and would benefit from a free surveyor's office tool, and **surveying students at the University of Nairobi, JKUAT, and the Technical University of Kenya** who want to learn modern surveying software and are willing to test in exchange for early access and a mention in the acknowledgments. The compensation is a free perpetual MetaRDU license when v2.0 launches (which costs nothing to grant) and public contributor credit in the release notes.

The feedback collection is structured exactly as in a paid beta: a daily 5-minute pulse survey via Google Forms (free), a weekly 30-minute deep-dive survey, and a final 60-minute interview. Weekly office hours are held on Zoom (free tier, 40-minute meetings) or Jitsi Meet (free, unlimited) every Thursday evening East Africa Time. The feedback is aggregated into a Beta Dashboard built in Notion (free) or a simple Markdown file in the repository, tracking Net Promoter Score, workflow completion rates, bug severity distribution, and feature request priority. The Reality Checker uses this dashboard as evidence at the midpoint gate and the pre-launch gate. The target is 5 surveyors in Month 5 and 10 surveyors in Month 6, recruited through ISK's WhatsApp groups, the `r/Surveying` subreddit, and direct outreach to surveying firms in Nairobi. Total cost: \$0.

9.4 Bug Bounty Program (Community-Based)

The bug bounty program launches in Phase 1 Month 3 on GitHub Security Advisories, which is free and integrated with the existing GitHub repository. The scope is the IPC boundary (any channel that can be invoked from the renderer with malformed input), the drone-data ingestion paths (KMZ, ODM output, MAVLink telemetry), the auto-update channel (manifest tampering, binary replacement), and the

cryptographic seal (RSA-2048 key handling). Out of scope: the renderer JavaScript (sandboxed), the Rust sidecar internal logic (not directly exposed), and social engineering. Because there is no cash reward pool, the rewards are non-cash: public acknowledgment in the release notes and Hall of Fame, a contributor credit in the repository, and a free MetaRDU Pro license when a paid tier launches. Most security researchers who report on open-source projects do so for the credit and the portfolio value, not for the cash.

The program is also listed on **Huntr.dev**, a free bug bounty platform specifically for open-source projects that handles triage and disclosure at no cost to the maintainer. Huntr.dev has a community of researchers who actively test OSS projects for free, and the platform provides a managed disclosure workflow that complies with CVE numbering and responsible disclosure timelines. This is a significantly better option than self-managing reports via GitHub Security Advisories alone, and it costs nothing. The combination of GitHub Security Advisories (for direct reports) and Huntr.dev (for community discovery) provides comprehensive coverage without any cash outlay.

9.5 Tutorial Videos (DIY) and Public Release

Three tutorial videos are produced in Phase 3 Month 8, each approximately 10 minutes long, covering the three core workflows: cadastral survey (12-leg traverse to sealed Form No. 4 PDF), topographic survey (50,000-point dataset to DXF with contours), and drone survey (flight plan to orthophoto to GCP verification to ML building footprint extraction). The videos are recorded in OBS Studio (free, open-source) at 1920 by 1080 resolution with audio from any USB microphone or the laptop's built-in mic. The editing is done in DaVinci Resolve (free version, professional-grade) or CapCut (free, simpler). The lead engineer records and edits the videos personally, which takes approximately 8 hours per video including script writing, recording, editing, and review. Total cost: \$0, plus 24 hours of the lead engineer's time.

The public v2.0.0 release in Month 9 follows a documented release runbook: freeze the main branch, run the full CI suite on all three platforms (free via GitHub Actions for public repos), run the Reality Checker pre-launch gate (requires evidence per criterion), tag the v2.0.0 commit, push the tag, let GitHub Actions build and upload the platform-specific binaries to GitHub Releases, publish the release notes, update the download page on Cloudflare Pages (free) or GitHub Pages (free), publish the three tutorial videos to YouTube (free), and announce on Twitter/X (free), the ISK mailing list (free), and the r/Surveying subreddit (free). The staged rollout begins at 1% on day 1, expands to 10% on day 7 if crash-free rate is above 99.5%, and expands to 100% on day 14. Crash monitoring uses Sentry's free Developer tier (5,000 errors per month), which is sufficient for a beta release with a few hundred users.

Chapter 10: Risk Matrix & Mitigation

Ten risks are identified, scored on likelihood and impact, and paired with a mitigation and a contingency. The risk heat map below shows the distribution: two risks are High/High (R1 Tauri migration breaks IPC handlers, R9 scope creep), three are Med/High (R2 ODM too heavy, R6 cert delays, R7 rusqlite migration), three are Med/Med (R3 MAVLink latency, R4 ML false positives, R8 GDAL bindings), and two are Low or Med/Med (R5 beta UX issues, R10 Reality Checker blocks). The overall risk profile is moderate: the highest risks (R1, R7) are mitigated by maintaining a parallel Electron release branch throughout Phase 3, so the team can roll

back to v1.x if the Tauri build fails to stabilize.

#	Risk	Likelihood	Impact	Mitigation	Contingency
R1	Tauri migration breaks 118 IPC handlers	High	High	Maintain parallel Electron branch; migrate one namespace at a time with E2E coverage	Delay v2.0 by 1 month; ship Tauri as v2.1
R2	ODM sidecar too heavy for low-end machines	Med	High	Gate feature behind 16GB RAM check; provide external WebODM fallback	Defer in-app photogrammetry to v2.1
R3	MAVLink telemetry latency on poor networks	Med	Med	Use USB serial not UDP; buffer telemetry at 5Hz; document USB cable requirements	Mark live telemetry as beta; prioritize mission export
R4	ML model false positives in feature extraction	Med	Med	Use IoU > 0.65 threshold; manual review UI before saving polygons	Ship building footprint only; defer road and change detection
R5	Beta uncovers UX issues blocking release	High	Med	Weekly office hours; structured feedback; iterate weekly on top 3 issues	Add 1-month buffer; slip to v2.0.1 patch releases
R6	Code-signing infrastructure setup (SignPath application, self-signed fallback)	Med	Med	Apply to SignPath Foundation in Month 1 (free, 1-2 weeks); self-signed cert as fallback with documented SmartScreen bypass; macOS ships unsigned with right-click Open docs	Ship unsigned with explicit warning on all platforms; defer signing to when revenue allows Apple Developer ID (\$99/yr)
R7	better-sqlite3 to rusqlite data migration loses data	Low	Critical	Write migration script with checksum verification; test on 10 sample databases; backup before migration	Maintain Electron branch indefinitely; cancel Tauri migration
R8	GDAL native bindings cross-platform issues	Med	Med	Use gdal-async with pre-built binaries; shell-out fallback; test on all 3 OS in CI	Use shell-out only; defer native bindings to v2.1
R9	Scope creep delays v2.0 beyond 9 months	High	High	Sprint Prioritizer agent; weekly scope review; defer non-P0 features to v2.1	Slip to 10-11 months; cut ML feature extraction from v2.0
R10	Reality Checker blocks release at pre-launch	Med	High	Run Reality Checker at midpoint too; weekly evidence collection; address issues as they arise	Address flagged issues; 2-4 week slip; v2.0.0-rc1 to closed beta

10.1 Top Three Risks in Detail

R1 Tauri migration breaks IPC handlers. This is the single largest risk because the migration touches every one of the 118 IPC handlers. The mitigation is to migrate one IPC namespace at a time, with full Playwright E2E coverage of every channel before the next namespace starts. The parallel Electron release branch is maintained throughout Phase 3, so if any namespace fails to stabilize on Tauri, the team can ship the Tauri

build with that namespace still routed through a sidecar Electron process (a hybrid deployment that adds complexity but unblocks release). The contingency is to delay v2.0 by one month and ship the Tauri build as v2.1, with v2.0 being a final Electron release that includes all the Phase 1 and 2 drone features.

R7 better-sqlite3 to rusqlite data migration. Although the likelihood is low, the impact is critical because data loss in a surveyor's project database is unacceptable. The migration script reads the existing SQLite database, creates a new rusqlite-backed database, copies all tables and rows, and verifies with a row count and a checksum comparison. The script is tested on 10 sample databases including the largest known MetaRDU project (50,000 points, 200 traverse legs, 30 GCPs). The backup strategy is to copy the original .metardu file to a .metardu.backup before running the migration, and to never delete the backup until the user explicitly confirms the migration succeeded. If the migration fails on any database, the Tauri build is pulled and the Electron branch is maintained indefinitely.

R9 Scope creep delays v2.0 beyond 9 months. The Sprint Prioritizer agent runs a weekly scope review with the lead engineer, comparing the actual progress against the planned milestones. Any feature that is more than 1 week behind schedule is flagged for either descope or defer. The default action is to defer non-P0 features to v2.1, with v2.0 shipping only the P0 features (Tauri migration, flight planning, live drone link, in-app photogrammetry, GDAL bindings) and the P1 features that are on track. The contingency is to slip the v2.0 release to 10 or 11 months, or to cut the ML feature extraction (the largest single Phase 3 workstream after Tauri migration) and ship it as v2.1.

Chapter 11: Budget, Timeline & KPIs

The total budget for the nine-month upgrade is \$0 to \$109 USD per year, depending on whether you choose to purchase the Apple Developer ID. Every other line item in the original \$15,000 to \$20,000 budget has a free equivalent that is equally production-ready. The principle is that the original budget assumed a paid GIS consultant, paid video editor, paid beta stipends, paid bug bounty rewards, paid code-signing certificates, paid Sentry, and paid drone hardware. Each of these has a free alternative: the GIS consultant is replaced by the agency-agents personas plus community Q&A on r/Surveying and GIS StackExchange; the video editor is replaced by the lead engineer using OBS Studio and DaVinci Resolve; the beta stipends are replaced by volunteer participation from surveyors who want early access; the bug bounty rewards are replaced by public acknowledgment and contributor credit; the code-signing certificates are replaced by SignPath Foundation for Windows and self-signing for macOS and Linux; Sentry is replaced by its free Developer tier; and the drone hardware is replaced by the ArduPilot SITL simulator plus a partnership with a local drone surveying firm.

11.1 Budget Breakdown (Zero-Cost Path)

Category	Item	Cost (USD/yr)	Free Alternative Used
Personnel	Lead engineer (you), 9 months full-time	\$0	Sweat equity; this is the entire investment

Category	Item	Cost (USD/yr)	Free Alternative Used
Personnel	GIS consultant	\$0	Replaced by agency-agents personas + r/Surveying + GIS StackExchange
Personnel	Video editor	\$0	DIY with OBS Studio (free) + DaVinci Resolve (free)
Infrastructure	GitHub (public repo)	\$0	Free for public repos; unlimited CI minutes for OSS
Infrastructure	Sentry error monitoring	\$0	Free Developer tier: 5,000 errors/month
Infrastructure	Domain + hosting	\$0 to \$10	GitHub Pages (free) or Cloudflare Pages (free); .com domain optional at \$10/yr via Cloudflare
Code-signing	Windows (SignPath Foundation)	\$0	Free for OSS projects; 1-2 week approval
Code-signing	Windows fallback (self-signed)	\$0	SmartScreen warning documented in README
Code-signing	macOS (Apple Developer ID)	\$0 or \$99	OPTIONAL: \$99/yr for notarization; OR free with documented right-click Open bypass
Code-signing	Linux (GPG self-signed)	\$0	Standard Linux distribution model
Auto-update	GitHub Releases hosting	\$0	Free for public repos
Hardware	Drone for testing	\$0	ArduPilot SITL simulator (free, tests all MAVLink code); DJI Assistant 2 simulator (free, tests DJI waypoint formats)
Hardware	Drone partnership	\$0	Partner with local surveying firm: they provide hardware + test data, you provide free MetaRDU licenses
Beta	Surveyor stipends	\$0	Volunteer participation; compensated with free perpetual license + contributor credit
Bug bounty	Cash rewards	\$0	GitHub Security Advisories (free) + Huntr.dev (free); rewards are public acknowledgment + Hall of Fame
TOTAL	Zero-cost path (no Apple Developer ID)	\$0/yr	All free alternatives; SmartScreen + Gatekeeper warnings documented
TOTAL	Minimum-viable paid path	\$10-109/yr	Domain (\$10) + optional Apple Developer ID (\$99)

11.2 Trade-offs of the Zero-Cost Path

The zero-cost path is not free in the sense of zero effort; it is free in the sense of zero cash. The trade-offs are: (1) the lead engineer spends more time on infrastructure setup (SignPath application, SmartScreen documentation, video editing) that a paid budget would outsource; (2) Windows users see a SmartScreen warning on first install, which reduces conversion by approximately 10-15% until reputation builds (typically 2-3 months after launch with steady download volume); (3) macOS users must right-click and Open the app on first launch, which is documented but adds friction; (4) the bug bounty program receives fewer reports than a paid program, because cash rewards attract professional researchers who would otherwise skip open-source projects; (5) the beta program may take longer to recruit volunteers than paid participants, though the

feedback quality is often higher because volunteers are genuinely interested in the product.

None of these trade-offs are release-blockers. Hundreds of successful open-source desktop apps, including Inkscape, Audacity, OBS Studio, and VS Code in its early days, shipped with these same trade-offs and grew into mainstream adoption. The key is to treat the paid budget items as upgrade targets: once MetaRDU generates revenue (through a Pro tier, consulting, or grants), the first \$99 pays for the Apple Developer ID (eliminating the macOS friction), the next \$300 pays for a Windows EV cert or Azure Trusted Signing subscription (eliminating the SmartScreen warning), and the next \$2,000 funds a bug bounty reward pool. Each paid upgrade directly improves conversion and security posture, and each can be justified by the revenue it unlocks.

11.3 Timeline (Gantt-Style)

The nine-month timeline is structured as three phases of three months each, with monthly milestones and phase gates. The table below summarizes the major milestones by month. Phase 1 (Months 1-3) focuses on stabilization and the flight planning engine. Phase 2 (Months 4-6) focuses on live drone connectivity and in-app photogrammetry, culminating in the closed beta and the Reality Checker midpoint gate. Phase 3 (Months 7-9) focuses on the Tauri migration, ML feature extraction, and the public v2.0 release, culminating in the Reality Checker pre-launch gate and the staged rollout.

Month	Phase	Major Milestones	Gate
M1	P1	zod schemas complete; 2 P0 math features; SignPath application submitted	Code Reviewer
M2	P1	5 P0 math features; flight planning engine; GDAL bindings	Code Reviewer
M3	P1	5 mission export formats; SignPath approved; bug bounty on Huntr.dev	GIS QA Engineer
M4	P2	MAVSDK-Rust sidecar; live telemetry dashboard; SITL testing	Code Reviewer
M5	P2	ODM sidecar; closed beta cohort 1 (5 volunteer surveyors)	Code Reviewer
M6	P2	Closed beta cohort 2 (10 surveyors); Playwright E2E	Reality Checker (midpoint)
M7	P3	Tauri shell scaffolding; 30 IPC handlers migrated	Code Reviewer
M8	P3	All 118 IPC handlers migrated; ML building footprints; DIY tutorial videos	GIS QA + Security
M9	P3	Public v2.0.0 release; staged rollout begins	Reality Checker (pre-launch)

11.4 KPI Dashboard

The KPI dashboard is reviewed weekly by the lead engineer, and biweekly by the Reality Checker (when active). The dashboard is a single Markdown file in the repository (docs/KPI_DASHBOARD.md) updated automatically by CI on every push. The metrics are grouped into four categories: engineering (binary size, idle memory, cold start, test pass rate, IPC coverage), product (beta NPS, workflow completion rates, bug severity

distribution), release (code-signing verification, auto-update success rate, crash-free sessions), and compliance (ASPRS class, ISO 19157 completeness, RDM 1.1 closure). The table below shows the target and current values for the top 10 KPIs as of the start of Phase 1.

KPI	Category	Target	Current (v1.0)
Binary size (Windows)	Engineering	< 15 MB	~150 MB
Idle memory	Engineering	< 120 MB	~350 MB
Cold start (M1 MacBook Air)	Engineering	< 1.2 s	~2.4 s
Engine test pass rate	Engineering	100%	100% (1259/1259)
IPC handler zod coverage	Engineering	100%	0%
Playwright E2E coverage	Engineering	>= 80% of journeys	0%
Beta surveyor NPS (volunteers)	Product	>= 7 of 10 score > 8	N/A
Crash-free sessions (7-day)	Release	>= 99.5%	Not measured
Auto-update success rate	Release	>= 99% over 1 minor bump	Never tested
ASPRS Class I compliance	Compliance	RMSE _{x,y} < 7.5 cm	Manual input only

Chapter 12: Conclusion & Next Actions

MetaRDU Desktop v2.0 is an ambitious but achievable upgrade that transforms the product from a surveyor's office tool into a true drone survey workstation, and it can be delivered for \$0 to \$109 per year. The three-phase approach manages risk by deferring the largest architectural change (Tauri migration) to Phase 3, after the Rust sidecar pattern is proven on the existing Electron shell. The five drone capabilities (flight planning, live drone link, in-app photogrammetry, real raster I/O, ML feature extraction) close the gap between the metardu brand promise and the actual product functionality. The six math standards (ASPRS 2014, NMAS 1947, ISO 19157, RDM 1.1 / Cap. 299, FGDC-STD-007.3, ICSM SP1) position the product for both the current Kenyan market and future international expansion. The production readiness workstream (SignPath Foundation code-signing, volunteer closed beta, community bug bounty, DIY tutorial videos, staged rollout on GitHub Releases) ensures that v2.0 is not just feature-complete but truly production-ready, without requiring any upfront capital.

The twelve-agent roster from the agency-agents framework provides the specialized expertise that a solo developer cannot afford to hire, with the Reality Checker serving as the quality gate at midpoint and pre-launch. The sequential-handoff workflow ensures that every agent's output is fully consumed by the next agent, preventing the context loss that plagues parallel multi-agent systems. The testing methodology (deterministic Playwright E2E, no sleeps, trace-on-retry, GIS QA gates) and the security model (IPC as trust boundary, narrowest-verb, zod validation per channel, SAST/DAST in CI) provide the engineering discipline needed for a production-grade desktop application distributed to field crews on unreliable networks. These are free methodologies that produce paid-tier quality.

The total budget of \$0 to \$109 per year is not a compromise; it is a legitimate production-readiness strategy that has been proven by hundreds of successful open-source desktop applications. The trade-offs are well-understood: SmartScreen warnings on Windows for the first 2-3 months, Gatekeeper bypass on macOS, and volunteer-based beta feedback instead of paid stipends. None of these are release-blockers. The key insight is that code-signing certificates, paid bug bounties, and paid beta stipends are optimization levers, not prerequisites. Once MetaRDU generates revenue, each paid upgrade directly improves conversion and security posture, and each can be justified by the revenue it unlocks. The first \$99 (Apple Developer ID) eliminates the macOS friction; the next \$300 (Windows EV cert or Azure Trusted Signing) eliminates the SmartScreen warning; the next \$2,000 funds a bug bounty reward pool. Every dollar spent after launch is a dollar earned back through higher conversion.

12.1 Top Five Next Actions (Next 30 Days, Zero Cost)

The following five actions should be taken in the next 30 days to kick off Phase 1. Each action is concrete, has a clear owner, costs \$0, and has a measurable completion criterion. These actions are the minimum viable start to the upgrade: completing them in the first month puts the project on track for the Phase 1 milestone cadence and the Month 3 phase gate.

Action 1: Apply to SignPath Foundation (free Windows code-signing). Visit signpath.org/foundation, submit the application for the metardu-desktop repository (MIT licensed, public on GitHub), and provide the project description and release history. Approval takes 1-2 weeks. While waiting, generate a self-signed certificate via PowerShell's New-SelfSignedCertificate cmdlet as a fallback, and write the SmartScreen bypass documentation (with screenshots) for the README and the future download page. Owner: lead engineer. Completion criterion: SignPath application submitted, self-signed cert generated, SmartScreen bypass docs committed to docs/INSTALL.md.

Action 2: Scaffold the Rust sidecar repository. Create a new packages/metardu-sidecar/ workspace in the monorepo with a Cargo.toml, a src/main.rs that reads length-prefixed JSON from stdin and writes length-prefixed JSON to stdout, and a simple ping/pong handler. Set up cross-compilation for Windows, macOS, and Linux in GitHub Actions CI (free for public repos). Owner: lead engineer. Completion criterion: the sidecar builds on all three platforms via CI and the ping/pong round-trip works from a test Electron handler.

Action 3: Implement camera-footprint math and KMZ export as proof of concept. In @metardu/engine, add a new flight-planning/ module with the camera footprint formulas ($GSD = \text{pixel_size} \times \text{altitude} / \text{focal_length}$, footprint width, footprint height, line spacing, photo spacing) and a KMZ export function that produces a DJI Pilot-compatible wpml file. Add property-based tests with fast-check that verify the math against known camera sensors (DJI Mavic 3, Phantom 4 RTK, senseFly eBee X) using published spec sheets. Owner: lead engineer, with the Drone/Reality Mapping agent persona from agency-agents as a reference. Completion criterion: the math produces correct GSD within 1% of spec on the test sensor database, and the KMZ file structure validates against the DJI wpml schema.

Action 4: Recruit 5 volunteer closed-beta surveyors. Reach out to the Institution of Surveyors of Kenya (ISK) WhatsApp groups, the r/Surveying subreddit, and surveying students at the University of Nairobi, JKUAT, and the Technical University of Kenya. Offer a free perpetual MetaRDU license and public

contributor credit in exchange for a 2-week trial in Months 5-6. Sign simple NDAs (use a free template from docracy.com or docsketch.com). Owner: lead engineer. Completion criterion: 5 signed NDAs and a confirmed calendar for the Month 5 trial.

Action 5: Run the Codebase Onboarding Engineer agent on metardu-desktop. Use the agency-agents Codebase Onboarding Engineer persona to produce a factual map of the current architecture, the IPC surface, the dependencies, and the tech-debt inventory. This map is the input to every subsequent agent and the foundation for ADRs 006-012. The agent persona is a free markdown file that you install into your AI coding assistant (Claude Code, Cursor, GitHub Copilot, etc.). Owner: lead engineer. Completion criterion: the architecture map is committed to `docs/onboarding-report.md`.

Call to Action (Zero Cost)

Start with Action 1 (SignPath Foundation application) today, because approval takes 1-2 weeks and there is no reason to wait. While waiting, complete Action 2 (Rust sidecar scaffold) and Action 3 (camera-footprint proof of concept) in parallel, both of which require only your time and a computer. Action 4 (volunteer beta recruitment) and Action 5 (codebase onboarding) can begin in Week 2. The goal of the first 30 days is to have the code-signing infrastructure in progress, the Rust sidecar pattern proven, and the flight planning math validated, all for \$0. None of these actions require any cash outlay. The only paid item in the entire plan is the optional \$99 Apple Developer ID, and that can wait until MetaRDU generates its first dollar of revenue.